

STATS_506_PS1

Zixun Zhang

Problem 1 - Exploratory Analysis of Soybean Cultivars

a. Import the data into a `data.frame` in R. Use the dataset documentation to assign appropriate variable names.

```
soybean <- read.csv("data.csv")

names(soybean) <- c(
  "season",
  "cultivar",
  "rep",
  "height",
  "first_pod",
  "stems",
  "legumes",
  "grains",
  "grains_pod",
  "seed_weight",
  "yield"
)

head(soybean)
```

	season	cultivar	rep	height	first_pod	stems	legumes	grains	grains_pod
1	1	NEO 760 CE	1	58.80	15.20	98.2	177.80	1.81	5.2
2	1	NEO 760 CE	2	58.60	13.40	102.0	195.00	1.85	7.2
3	1	NEO 760 CE	3	63.40	17.20	100.4	203.00	2.02	6.8
4	1	NEO 760 CE	4	60.27	15.27	100.2	191.93	1.89	6.4
5	1	MANU IPRO	1	81.20	18.00	98.8	173.00	1.75	7.4
6	1	MANU IPRO	2	75.80	20.80	69.2	128.00	1.85	7.2
	seed_weight	yield							
1	152.20	3232.82							
2	141.69	3517.36							
3	148.81	3391.46							
4	148.50	3312.58							

```
5      145.59 3230.99
6      154.87 3374.80
```

b. Using the information provided in the dataset documentation: Verify that there are 320 observations in the dataset; 40 distinct cultivars; and all the observations were collected from two seasons. Produce a table showing the number of observations from each season.

```
obs_num <- nrow(soybean)

cul_num <- length(unique(soybean$cultivar))

seasons <- unique(soybean$season)

obs_num
```

```
[1] 320
```

```
cul_num
```

```
[1] 40
```

```
seasons
```

```
[1] 1 2
```

```
table(soybean$season)
```

```
 1  2
160 160
```

c. Use the data to answer the following questions:

1. What is the correlation between plant height (PH) and grain yield (GY)?
2. Calculate the correlation between PH and GY separately for each season. Which season has the higher correlation?
3. What is the grain yield of the soybean observation with the greatest plant height?

4. What percentage of observations have a thousand seed weight (MHG) greater than the overall average thousand seed weight?
5. Which cultivar has the highest average grain yield?

```
# 1
print("Correlation between plant height and grain yield:")
```

```
[1] "Correlation between plant height and grain yield:"
```

```
cor(soybean$height, soybean$yield)
```

```
[1] 0.1232807
```

```
# 2
print("Correlation between plant height and grain yield for season 1:")
```

```
[1] "Correlation between plant height and grain yield for season 1:"
```

```
cor(soybean$height[soybean$season == 1],
    soybean$yield[soybean$season == 1])
```

```
[1] 0.2308581
```

```
print("Correlation between plant height and grain yield for season 2:")
```

```
[1] "Correlation between plant height and grain yield for season 2:"
```

```
cor(soybean$height[soybean$season == 2],
    soybean$yield[soybean$season == 2])
```

```
[1] -0.0227046
```

```
# 3
print("Grain yield of the soybean observation with the greatest plant height")
```

```
[1] "Grain yield of the soybean observation with the greatest plant height"
```

```
soybean$yield[which.max(soybean$height)]
```

```
[1] 3304.678
```

```
# 4
print("Percentage of observations have a thousand seed weight (MHG) greater than
the overall average")
```

```
[1] "Percentage of observations have a thousand seed weight (MHG) greater than
the overall average"
```

```
mean(soybean$seed_weight > mean(soybean$seed_weight)) * 100
```

```
[1] 47.1875
```

```
# 5
print("Cultivar with the highest average grain yield:")
```

```
[1] "Cultivar with the highest average grain yield:"
```

```
avg_yield <- tapply(soybean$yield, soybean$cultivar, mean)
names(avg_yield)[which.max(avg_yield)]
```

```
[1] "98R30 CE"
```

1. The correlation between plant height and grain yield is about 0.1233.
2. The correlation between plant height and grain yield for season 1 is about 0.2309, and about -0.0227 for season 2.
3. The grain yield of the soybean observation with the greatest plant height is 3304.678 kg/ha.
4. About 47.19% observations have a thousand seed weight greater than the overall average thousand seed weight.
5. "98R30 CE" has the highest average grain yield.

d. Create a table showing the average value of each quantitative variable:

1. One row containing overall averages.

2. One row for each season containing season-specific averages.

```
quant <- soybean[, c(
  "height", "first_pod", "stems", "legumes",
  "grains", "grains_pod", "seed_weight", "yield"
)]

overall <- sapply(quant, mean)

season1 <- sapply(quant[soybean$season == 1, ], mean)
season2 <- sapply(quant[soybean$season == 2, ], mean)

avg_table <- rbind(
  Overall = overall,
  Season1 = season1,
  Season2 = season2
)

avg_table
```

	height	first_pod	stems	legumes	grains	grains_pod	seed_weight
Overall	68.38678	15.465	59.08831	135.0858	2.290844	4.071656	168.3223
Season1	65.98506	16.485	57.96506	135.2350	2.299688	4.816625	162.2888
Season2	70.78850	14.445	60.21156	134.9367	2.282000	3.326687	174.3558
yield							
Overall	3418.554						
Season1	3428.452						
Season2	3408.655						

e. Carry out a series of t-tests to examine whether the grain yield (GY) differs between the two seasons.

1. State the null and the alternative hypotheses.
2. Present the R output and interpret the results.
3. The null hypothesis would be the grain yields are the same between the two seasons. The alternative hypothesis is that the grain yields are different between the two seasons.

```
season1_yield <- soybean$yield[soybean$season == 1]
season2_yield <- soybean$yield[soybean$season == 2]

t.test(season1_yield, season2_yield)
```

Welch Two Sample t-test

```
data: season1_yield and season2_yield
t = 0.35155, df = 231.85, p-value = 0.7255
alternative hypothesis: true difference in means is not equal to 0
95 percent confidence interval:
 -91.15712 130.75170
sample estimates:
mean of x mean of y
3428.452 3408.655
```

2. Assuming the grain yields are the same between the two seasons, the probability of observing a t statistics of 0.352 or more further away on either direction is 0.7255. Using a significance level of 5%, the data do not provide enough evidence to conclude that the grain yields are different across the two seasons.

Problem 2 - Adult Income Dataset

a. Import the data into a **data.frame** in R. As with the previous data, you may download the data outside of your submission, but importation should take place inside the problem set submission.

```
adult <- read.csv("adult.data", header = FALSE)
head(adult)
```

	V1	V2	V3	V4	V5	V6			
1	39	State-gov	77516	Bachelors	13	Never-married			
2	50	Self-emp-not-inc	83311	Bachelors	13	Married-civ-spouse			
3	38	Private	215646	HS-grad	9	Divorced			
4	53	Private	234721	11th	7	Married-civ-spouse			
5	28	Private	338409	Bachelors	13	Married-civ-spouse			
6	37	Private	284582	Masters	14	Married-civ-spouse			
	V7	V8	V9	V10	V11	V12	V13	V14	
1	Adm-clerical	Not-in-family	White	Male	2174	0	40	United-States	
2	Exec-managerial	Husband	White	Male	0	0	13	United-States	
3	Handlers-cleaners	Not-in-family	White	Male	0	0	40	United-States	
4	Handlers-cleaners	Husband	Black	Male	0	0	40	United-States	
5	Prof-specialty	Wife	Black	Female	0	0	40	Cuba	
6	Exec-managerial	Wife	White	Female	0	0	40	United-States	
	V15								
1	<=50K								
2	<=50K								

```

3  <=50K
4  <=50K
5  <=50K
6  <=50K

```

b. Clean and simplify the variable names.

```

names(adult) <- c(
  "age",
  "work_class",
  "final_weight",
  "edu",
  "yr_of_edu",
  "marital",
  "occupation",
  "relationship",
  "race",
  "sex",
  "gain",
  "loss",
  "hr_per_week",
  "native_country",
  "income"
)

head(adult)

```

	age	work_class	final_weight	edu	yr_of_edu	marital
1	39	State-gov	77516	Bachelors	13	Never-married
2	50	Self-emp-not-inc	83311	Bachelors	13	Married-civ-spouse
3	38	Private	215646	HS-grad	9	Divorced
4	53	Private	234721	11th	7	Married-civ-spouse
5	28	Private	338409	Bachelors	13	Married-civ-spouse
6	37	Private	284582	Masters	14	Married-civ-spouse

	occupation	relationship	race	sex	gain	loss	hr_per_week
1	Adm-clerical	Not-in-family	White	Male	2174	0	40
2	Exec-managerial	Husband	White	Male	0	0	13
3	Handlers-cleaners	Not-in-family	White	Male	0	0	40
4	Handlers-cleaners	Husband	Black	Male	0	0	40
5	Prof-specialty	Wife	Black	Female	0	0	40
6	Exec-managerial	Wife	White	Female	0	0	40

	native_country	income
1	United-States	<=50K

```

2 United-States <=50K
3 United-States <=50K
4 United-States <=50K
5           Cuba <=50K
6 United-States <=50K

```

c. Examine missing values. Report the number of missing observations for each variable.

```

sapply(adult, function(x){
  sum(x == " ?")
})

```

age	work_class	final_weight	edu	yr_of_edu
0	1836	0	0	0
marital	occupation	relationship	race	sex
0	1843	0	0	0
gain	loss	hr_per_week	native_country	income
0	0	0	583	0

d. Restrict the sample to individuals working at least 10 hours per week. Confirm the sample size before and after filtering.

```
print("Sample size before filtering:")
```

```
[1] "Sample size before filtering:"
```

```
nrow(adult)
```

```
[1] 32561
```

```
print("Sample size after filtering:")
```

```
[1] "Sample size after filtering:"
```

```

working_adult <- adult[adult$hr_per_week >= 10, ]
nrow(working_adult)

```

```
[1] 32103
```

e. Remove observations with missing values for occupation, workclass, or native country. Explain why.

```
missing_data <- " ?"

working_adult <- working_adult[
  working_adult$occupation != missing_data &
  working_adult$work_class != missing_data &
  working_adult$native_country != missing_data,
]
```

We are removing observations with missing values because they do not provide complete information for the subsequent analysis.

f. Eliminate any observations with: age less than 18, age greater than 90, education level identified as Preschool.

```
working_adult <- working_adult[
  working_adult$age >= 18 &
  working_adult$age <= 90 &
  working_adult$edu != "Preschool",
]
```

g. Report your final sample size.

```
print("The final sample size is:")
```

```
[1] "The final sample size is:"
```

```
nrow(working_adult)
```

```
[1] 29568
```

The final sample size is 29568.

Problem 3 - Happy Numbers

a. Write a function `nextHappy()` that given a positive integer, computes the next value in the happy-number sequence. Be sure to provide a reasonable error on an invalid input. Be sure to document your function

```
#' Calculate the next happy number.
#'
```

```

#' Computes the next value in a happy-number sequence by summing
#' the squares of the digits of a positive integer.
#'
#' @param n A positive integer.
#'
#' @returns The sum of the squared digits of `n`.
nextHappy <- function(n) {
  if (!is.numeric(n) || length(n) != 1) {
    stop("n must be a single numeric value")
  }
  if (n <= 0 || n %% 1 != 0) {
    stop("n must be a positive integer")
  }

  total <- 0

  while (n > 0) {
    digit <- n %% 10
    total <- total + digit^2
    n <- n %/% 10
  }

  return(total)
}

```

```
nextHappy(19)
```

```
[1] 82
```

```
nextHappy(82)
```

```
[1] 68
```

b. Create a function `happySequence()` that returns the sequence beginning at a given number. Use your `nextHappy()` function to perform the calculation. Be sure to provide a reasonable error on an invalid input. Be sure to document your function

```

#' Identify if a number is a happy number.
#'
#' Determines whether a positive integer is a happy number. If the number is
#' happy, the happy-number sequence leading to 1 is also returned.

```

```

#'
#' @param n A positive number
#'
#' @returns A list containing `isHappy`, a logical value indicating whether
#'   `n` is a happy number. If `n` is happy, the list also contains `sequence`,
#'   a numeric vector giving the sequence from `n` to 1.
happySequence <- function(n) {

  if (!is.numeric(n) || length(n) != 1) {
    stop("n must be a single numeric value")
  }

  if (n <= 0 || n %% 1 != 0) {
    stop("n must be a positive integer")
  }

  prev <- c()
  current <- n

  while (!(current %in% prev)) {

    prev <- c(prev, current)

    if (current == 1) {
      return(return(list(
        sequence = prev,
        isHappy = TRUE
      )))
    }
    current <- nextHappy(current)
  }

  return(list(
    isHappy = FALSE
  ))
}

```

```
happySequence(19)
```

```

$sequence
[1] 19 82 68 100 1

```

```
$isHappy  
[1] TRUE
```

```
happySequence(4)
```

```
$isHappy  
[1] FALSE
```

c. Among integers 1 through 1000:

- How many are happy?
- What percentage are happy?
- What starting value produces the longest happy sequence before terminating?

```
nums <- 1:1000  
  
happy_seq <- sapply(nums, function(x) {  
  happySequence(x)$isHappy  
})  
  
happy_count <- sum(happy_seq)  
happy_count
```

```
[1] 143
```

```
happy_percentage <- happy_count / length(nums) * 100  
happy_percentage
```

```
[1] 14.3
```

```
happy_seq_len <- sapply(nums, function(x) {  
  length(happySequence(x)$sequence)  
})  
longest_seq_val <- nums[which.max(happy_seq_len)]  
longest_seq_val
```

```
[1] 356
```

- a. 143 numbers are happy numbers
- b. 14.3% numbers are happy numbers
- c. 356 produces the longest happy sequence before terminating.

Github repository link: <https://github.com/zixunzhang99/stats506>